

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 642 — Deep Reinforcement Learning

The same Bellman equation as CSCE 625, with the model unknown and the table replaced by a network

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 7 (with CSCE 753 and CSCE 625)

Prerequisites: CSCE 625 Modules 09–10 for MDPs, CSCE 636 for deep networks, CSCE 669 Modules 05 and 08 for the optimisation; probability

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A working agent on a task you chose, trained across at least five random seeds, reported with variance, compared against a non-learned baseline, and accompanied by an honest statement of whether learning was the right approach

Course description

CSCE 625 Module 10 solved Markov decision processes by computation, with the model known and the value function in a table. This course removes both. The agent does not know the transition probabilities or the rewards, and the state space is far too large to tabulate — so it must learn from interaction and approximate with a network.

Every difficulty in this course traces to one of those two substitutions, which is the most useful organising fact available. Not knowing the model causes sample inefficiency and makes exploration a problem (Modules 02 and 08). Replacing the table with an approximator causes instability and destroys the convergence guarantees (Module 04).

The second theme is that reward specification is the hard part. CSCE 625 Module 01 made the point in the abstract; here it has teeth, because a reinforcement learning agent optimises its reward with far more ingenuity and less restraint than a search does. Module 10 is entirely about this, and it is the module with the most practical consequence.

The third is that this field has a serious reproducibility problem, and the course does not soften it. Module 12 is about variance across random seeds, which in reinforcement learning is routinely larger than the differences between published methods — a situation that makes most single-seed comparisons uninformative, and that you need to know before reading the literature.

And the closing position is scoped rather than enthusiastic. Module 13 states where reinforcement learning demonstrably works, where it does not, and how to tell which case you are in — because the honest answer is that CSCE 625's methods are the right choice far more often than this field's literature suggests, and knowing when to use neither is part of knowing the subject.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 State precisely what reinforcement learning adds to CSCE 625's MDP.
- 2 Explain the exploration-exploitation trade and the standard strategies.
- 3 Derive temporal-difference learning and distinguish on-policy from off-policy.
- 4 Explain the deadly triad and the mechanisms that stabilise deep Q-learning.
- 5 Derive the policy gradient and explain variance reduction.
- 6 Explain trust-region methods and why PPO is the default.
- 7 Choose an algorithm for continuous control and explain the entropy term.
- 8 Explain model-based methods and their sample-efficiency argument.
- 9 Explain offline reinforcement learning and why naive off-policy learning fails there.
- 10 Design a reward and anticipate how it will be gamed.
- 11 Evaluate an agent across seeds and report variance honestly.
- 12 State whether reinforcement learning is the right approach for a given problem.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	From Planning to Learning	What exactly changes when you do not know the model?
2	Exploration	How should an agent balance using what it knows against finding out more?
3	Learning Values from Experience	How do you improve a value estimate from a single transition?
4	Deep Q-Networks	Why is replacing a table with a network so much harder than it sounds?
5	Policy Gradients	What if you skip the value function and differentiate the objective?
6	Actor-Critic and Trust Regions	How do you get a value function's efficiency and a policy gradient's stability?

Wk	Module	Core question it answers
7	Continuous Control	What changes when the action is a vector of real numbers?
8	Model-Based Methods	Can you buy sample efficiency by learning a model?
9	Learning Without Interaction	What can you learn from data you did not collect?
10	Reward Design	How do you specify what you want?
11	Multiple Agents and Self-Play	What breaks when the other agents are also improving?
12	Reproducibility	How do you tell whether a reinforcement learning result is real?
13	Where It Works	After twelve modules, when should you actually use this?

Projects

Project 1 — Implement and diagnose (after Module 7)

Implement value-based and policy-gradient methods from scratch, and diagnose them rather than merely running them.

Requirements

- **Tabular Q-learning and SARSA** on a grid world, with the on-policy/off-policy difference demonstrated on the cliff-walking task.
- **DQN from scratch** on CartPole, with replay and a target network, and an ablation of each.
- **REINFORCE with and without a baseline**, with the gradient variance measured.
- **PPO from scratch**, matching a reference implementation's performance.
- **Five seeds for every configuration**, with the spread plotted rather than the mean alone.
- **A learning curve that fails**, diagnosed to a specific cause.

Done means

- **The cliff-walking difference reproduced and explained** — it is the clearest demonstration of on-policy against off-policy that exists.
- **Each DQN ablation's effect measured**, not asserted.
- **Gradient variance reported with and without the baseline**, as numbers.
- **Five seeds everywhere, with the spread shown**, which is the graded discipline.

Project 2 — An agent on your own task (after Module 12)

Apply reinforcement learning to a problem you chose, and establish honestly whether it was the right approach.

Requirements

- **A task you care about**, with the state, action, and reward design written out and justified.
- **A non-learned baseline** from CSCE 625 — scripted, planned, or searched.
- **At least five seeds**, reported with interquartile means and confidence intervals.
- **A reward-gaming attempt**: find a behaviour that scores well and is not what you wanted.
- **An evaluation under a changed environment**, not just the training one.
- **Sample count and wall-clock time** reported.

Done means

- **The baseline comparison honest, including if the baseline wins** — which is a complete and respectable result.

- **Seed variance reported properly**, by Module 12's standard.
- **At least one reward-gaming behaviour found and described**, because there is almost always one.
- **A written answer to 'was reinforcement learning the right tool here?'** — which is the graded part, and 'no' is an acceptable answer with evidence.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Sutton & Barto — Reinforcement Learning: An Introduction, 2nd edition (free PDF)	The primary source, free from the authors. Chapters 2, 5–7, 9–13 are Modules 02–06; chapters 3–4 were CSCE 625 Module 10, which is why the two courses share one book.
OpenAI Spinning Up in Deep RL (free)	The best free practical introduction — clean reference implementations of the Module 05–07 algorithms, with the derivations and an unusually honest discussion of what is hard.
Berkeley CS285 — Deep Reinforcement Learning (free lectures and homework)	The graduate course this one follows most closely. Full video lectures and assignments; Modules 05–09 track it.
Henderson et al. — Deep Reinforcement Learning That Matters (free)	Module 12's source. The seed-variance measurements that should change how you read every other paper in the field.
Agarwal et al. — Deep RL at the Edge of the Statistical Precipice (free)	How to report reinforcement learning results properly — Module 12's recommended practice, with the statistical argument and free tooling.
Gymnasium, Stable-Baselines3, and CleanRL (free)	The working toolkit. CleanRL's single-file implementations are the ones to read — every algorithm in one file, with the implementation details that papers omit.

Tooling

- **Gymnasium for environments, and start with CartPole and a tabular grid world rather than Atari — a bug in your algorithm is diagnosable in five seconds on CartPole and invisible for six hours on Atari.**
- **CleanRL as a reference, not a dependency. Read its PPO implementation line by line;** the differences between it and the paper are the things that actually matter.
- **A 4 GB GPU is adequate for this entire course. Reinforcement learning is bottlenecked on environment steps, not on gradient computation** — which is good news for the hardware and bad news for the wall-clock time.
- **Seed control and logging from the first line of code. Module 12 requires five seeds per configuration and retrofitting that is painful, so build it in at the start.**
- **Weights and Biases or TensorBoard, logging episode return, episode length, loss, gradient norm, and the entropy or exploration parameter. All five, from the beginning.**
- **Patience, and a non-learned baseline. Every experiment in this course should be compared against the CSCE 625 method for the same task, and sometimes the baseline wins.**

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.